

문제풀이 및 오답기능

1. 기능개요

1.1 기능구성

문제풀이 기능은 학습자가 레슨 단위로 문제를 풀고, 결과에 따라 통과 여부를 확인할 수 있도록 구성한 기능입니다. 한 세트는 10문제로 이루어지며, 7문제 이상 정답일 경우 해당 레슨의 문제풀이를 통과한 것으로 처리합니다.

오답 기능은 문제풀이 과정에서 틀린 문제를 별도로 저장하고, 이후 다시 풀기와 해설 확인을 통해 복습할 수 있도록 설계했습니다. 단순 채점에서 끝나는 것이 아니라 학습 흐름과 복습 흐름이 자연스럽게 이어지도록 구현한 것이 핵심입니다.

1.2 주요 특징

- 'lessonId' 기준으로 문제풀이 세트 생성
- 10문제 중 7문제 이상 정답 시 통과
- 문제 건너뛰기 기능 지원
- 완료 후 정답/오답/건너뛰기 수 요약 표시
- 오답노트 목록, 상세, 다시 풀기 기능 제공
- 오답 재정답 시 1회 보상 점수 부여
- 결과에 따라 다음 레슨, 레슨 목록, 학습 메인으로 이동 분기

2. 문제풀이 기능 구현

2.1 문제 세트 생성

문제풀이 시작 시 사용자가 선택한 레슨의 'lessonId'를 기준으로 문제를 조회하

고, 해당 세트에 대한 시도 기록을 생성합니다. 이때 'practice_set_attempts'에는 세트 단위 정보가 저장되고, 'practice_set_items'에는 문제 순서가 저장됩니다.

이 구조를 통해 단순히 문제를 보여주는 것이 아니라, 사용자가 어떤 레슨의 어떤 세트를 어떤 순서로 풀었는지를 추적할 수 있도록 했습니다.

2.2 답안 제출 및 채점

사용자가 답안을 제출하면 문제의 정답과 비교하여 정·오답 여부를 판정합니다. 정답인 경우 제출 기록과 함께 점수가 반영되고, 오답인 경우에는 오답노트 대상으로 저장됩니다.

또한 사용자가 문제를 바로 풀지 못할 경우 건너뛰기 기능을 사용할 수 있도록 구성했습니다. 건너뛴 문제는 별도 상태로 관리되며, 최종 제출 전에 다시 돌아가 재도전할 수 있도록 흐름을 설계했습니다.

2.3 완료 처리 및 이동 분기

세트 종료 시 정답 수를 기준으로 통과 여부를 판정하고, 통과한 경우에는 학습 진행 상태와 문제풀이 완료 상태를 함께 반영합니다. 완료 화면에서는 정답, 오답, 건너뛰기 수를 요약해서 보여주며, 사용자의 학습 상황에 따라 다음 이동 경로를 다르게 제공합니다.

예를 들어 통과 시에는 다음 레슨 또는 학습 메인으로 이동할 수 있고, 미통과 시에는 같은 레슨의 문제를 다시 풀거나 레슨 목록으로 돌아갈 수 있도록 분기했습니다.

3. 오답 복습 기능 구현

3.1 오답 목록 및 상세 조회

오답 목록은 전체 오답을 확인할 수 있을 뿐 아니라, 특정 레슨 기준으로 필터링하여 해당 레슨의 오답만 확인할 수 있도록 구현했습니다. 이를 통해 사용자는 현재 학습 중인 단원과 직접 연결된 복습 흐름을 유지할 수 있습니다.

오답 상세 화면에서는 문제 내용, 정답, 해설, 오답 횟수, 재시도 여부 등을 확인할 수 있도록 구성했습니다.

3.2 다시 풀기 및 보상 처리

오답 다시 풀기 기능에서는 사용자가 동일 문제에 다시 답안을 제출할 수 있으며, 정답일 경우 상태를 해결 완료로 변경합니다. 이때 이미 보상이 지급된 이력이 있는지 확인한 후, 최초 재정답에 한해서만 보상 점수를 부여하도록 처리했습니다.

이 과정에서 중복 점수 지급이 발생하지 않도록 idempotency key 개념을 반영해 안정적으로 동작하도록 설계했습니다.

3.3 구현 과정에서 고려한 점

문제풀이와 오답 기능은 서로 다른 화면처럼 보이지만 실제로는 하나의 학습 흐름으로 이어져야 했기 때문에, 'lessonId'를 기준으로 데이터 흐름이 끊기지 않도록 맞추는 작업이 중요했습니다.

또한 사용자가 잘못된 세트나 다른 사용자의 오답 기록에 접근하지 못하도록 예외 상황을 점검했고, 이동 경로가 꼬이지 않도록 레슨 목록, 오답 목록, 상세 화면 사이의 연결 관계도 함께 정리했습니다.

4. 결과 및 느낀 점

문제풀이와 오답 기능을 구현하면서 단순히 문제를 맞히고 틀리는 기능이 아니라, 학습자가 반복 학습과 복습을 자연스럽게 이어갈 수 있는 구조를 만드는 것이 중요하다는 점을 느꼈습니다.

특히 레슨 흐름, 오답 저장, 재시도 보상, 완료 후 이동 분기까지 연결하면서 사용자 경험과 데이터 구조를 함께 고려해야 했고, 이 과정을 통해 기능 단위 구현이 아니라 학습 흐름 전체를 바라보는 시각을 기를 수 있었습니다.

[주요코드-문제풀이]

PracticeService.java

```
public PracticeSetResponse completeSet(SessionUser user, Long setAttemptId)
{
    PracticeSetAttempt attempt = requireOwnedAttempt(user, setAttemptId);
    if (!"COMPLETED".equals(attempt.getStatus())) {
        throw new BusinessException(COMMON_IDEMPOTENCY_CONFLICT);
    }

    if (Boolean.TRUE.equals(attempt.getPassed())) {
        boolean firstCompletion = scoreService.giveScoreIfAbsent(...);
        if (firstCompletion) {
            progressService.markPracticePassed(userId, subjectId, nodeId);
            userLessonProgressMapper.upsertPracticePassed(userId, lessonId);
            attendanceService.recordAttendanceOnPracticePass(userId,
            setAttemptId);
        }
    }
    return nextStepResponse(attempt);
}
```

- ✓ 10문제 답안을 순회해 정답·건너뛸·오답을 분리하고 제출 이력을 먼저 저장
- ✓ 정답은 점수, 오답은 오답노트로 연결해 결과·복습·랭킹의 공통 근거를 만듦
- ✓ 세트 통과는 레슨 문제풀이·출석·보상을 한 transaction 흐름에서 반영
- ✓ 오답 재제출은 기존 제출을 갱신하고 idempotency key로 재정답 보상 중복차단

[관련화면-문제풀이]

The screenshot shows the Knowva web application interface. At the top, there's a navigation bar with the Knowva logo and several tabs: '학습' (Study), '분석' (Analysis), '커뮤니티' (Community), and '마이페이지' (My Page). On the right, there are icons for a moon and a user profile labeled '누' (You) and '누비학습자' (Study User). Below the navigation bar, the main content area is titled 'Practice' and 'Java 조건문 01 - 핵심 개념' (Java Conditional Statement 01 - Core Concept). Underneath the title, it says 'BRONZE Java 조건문 형성' (BRONZE Java Conditional Statement Formation). The question is labeled '문제 1 / 10' (Question 1 / 10) and is a 'MULTIPLE_CHOICE' type. The question text is 'Java 조건문 01 - 핵심 개념 문제 1. 핵심 개념으로 가장 알맞은 것을 고르세요.' (Java Conditional Statement 01 - Core Concept Question 1. Choose the most appropriate one based on the core concept). There are four radio button options: 'A 핵심 개념' (A Core Concept), 'B BRONZE 오답 보기' (B BRONZE Wrong Answer View), 'C 문제와 무관한 선택지' (C Irrelevant choice to the problem), and 'D 정답 없음' (D No correct answer). At the bottom of the question area, there are two buttons: '건너뛰기' (Skip) and '제출' (Submit).

[주요코드-오답복습]

WrongAnswerService.java

```
boolean isCorrect = normalizeAnswer (problem .getAnswerText ())
    .equals (normalizeAnswer (form .getSubmittedAnswer ()));

Optional <PracticeSubmission > existingSubmission =
    practiceSubmissionMapper .findBySetAttemptIdAndProblemIdAndContext (
        wrongAnswer .getSetAttemptId (),
        problem .getProblemId (),
        "WRONG_ANSWER_RETRY "
    );

if (existingSubmission .isPresent ()) {
    PracticeSubmission submission = existingSubmission .get();
    submission .setSubmittedAnswer (form .getSubmittedAnswer ());
    submission .setIsCorrect (isCorrect);
    practiceSubmissionMapper .updateSubmission (submission);
} else {
    PracticeSubmission submission = new PracticeSubmission ();
    submission .setSetAttemptId (wrongAnswer .getSetAttemptId ());
    submission .setUserId (sessionUser .userId ());
    submission .setProblemId (problem .getProblemId ());
    submission .setSubmissionContext ("WRONG_ANSWER_RETRY ");
    submission .setSubmittedAnswer (form .getSubmittedAnswer ());
    submission .setIsCorrect (isCorrect);
    practiceSubmissionMapper .insertSubmission (submission);
}
```

```
if (isCorrect) {
    String retryIdempotencyKey = "WRONG_ANSWER_RETRY :" + wrongAnswer .getWrongAnswerId ();

    wrongAnswer .setReviewStatus ("SOLVED");
    wrongAnswer .setRetryBonusAwarded (true);
    wrongAnswer .setLastSubmissionId (latestSubmissionId);

    int existingScoreEventCount = scoreEventMapper .countByIdempotencyKey (retryIdempotencyKey);

    if (existingScoreEventCount == 0) {
        scoreService .giveScore (
            sessionUser .userId (),
            problem .getSubjectId (),
            wrongAnswer .getWrongAnswerId (),
            "WRONG_ANSWER_RETRY ",
            5,
            "WRONG_ANSWER_RETRY_CORRECT ",
            retryIdempotencyKey
        );
    }
}
```

- ✓ 사용자가 오답 문제를 다시 제출하면 기존 제출 이력이 있는지 먼저 확인
- ✓ 기존 이력이 있으면 수정하고, 없으면 새 제출을 생성
- ✓ 정답일 경우 오답 상태를 'SOLVED'로 바꾸고 재정답 보상 지급 여부를 저장
- ✓ 'idempotency key'를 사용해 +5 점수가 중복 지급되지 않도록 방지

[관련화면-오답복습]

• 학습• 분석• 커뮤니티• 마이페이지🌙누누비학습자

Review

Java 조건문 01 - 핵심 개념

BRONZE Java 조건문 형성

학습 메인으로

오답 요약 보기

Java 조건문 01 - 핵심 개념
문제 10. 레슨 내용을 종합한 코드를 작성하세요.

레슨 ID: 201
문제 ID: 20110
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념
문제 9. 오류를 줄이는 검증 코드를 작성하세요.

레슨 ID: 201
문제 ID: 20109
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념
문제 8. 짧은 코드 조각을 작성하세요.

레슨 ID: 201
문제 ID: 20108
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념
문제 7. 자료를 다룰 때 필요한 표현을 입력하세요.

레슨 ID: 201
문제 ID: 20107
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념
문제 6. 조건 또는 반복 흐름의 핵심 단어를 입력하세요.

레슨 ID: 201
문제 ID: 20106
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념
문제 5. 빈칸에 들어갈 핵심 표현을 입력하세요.

레슨 ID: 201
문제 ID: 20105
오답 횟수: 1
상태: PENDING

상세 보기

Java 조건문 01 - 핵심 개념 문제 10. 레슨 내용을 종합한 코드를 작성하세요.

문제 ID: 20110

오답 횟수: 1

상태: SOLVED

재시도 보상 지급 여부: true

대단해요! 스스로 부족한 점을 찾아 채워가는 모습이 정말 멋집니다!

정답: `System.out.println(value);`

함께 살펴보기

해설: Java 조건문 01 - 핵심 개념의 레슨 내용을 종합한 코드를 작성하세요. 기준 정답은 `System.out.println(value);`입니다.

검토 완료

검토 완료 처리